

THEORY ASSIGNMENT

Subject: Data Structure

Unit – 2: Linear Data Structure

Course: BCA Semester – 2 (GTU)

Submission Date: 15th March 2026

Q.1: 1- MARK

1) Give the full form of LIFO & FIFO

- **LIFO** – *Last In First Out*
- **FIFO** – *First In First Out*

2) Explain Linked List.

A **Linked List** is a linear data structure in which elements (called nodes) are stored non-contiguously and each node contains data and a pointer to the next node.

3) Define Data Structure.

A **Data Structure** is a systematic way of organizing and storing data in memory so that it can be accessed and modified efficiently.

4) What is Primitive Data Types?

Primitive data types are the basic built-in data types provided by a programming language, such as **int**, **char**, **float**, **double**, etc.

5) What are some applications of Data Structure?

Data structures are used in:

- Searching and Sorting
 - Database Management Systems
 - Operating Systems
 - Compiler Design
 - Memory Management
-

6) How do you test for an empty stack?

A stack is empty when the **top pointer is equal to -1** (in array implementation).

7) List some common data structures.

- Array
 - Stack
 - Queue
 - Linked List
 - Tree
 - Graph
-

8) Full form of DS.

DS – Data Structure

9) Process of inserting an element in stack is called _____.

The process is called **Push**.

10) The concatenation of two lists can be performed in $O(1)$ time. Which variation of the linked list can be used?

Circular Linked List (because the last node directly connects to the first node).

11) How do you initialize an array in C?

An array can be initialized at the time of declaration as:

```
int arr[5] = {1, 2, 3, 4, 5};
```

12) A queue follows _____

A queue follows the **FIFO (First In First Out)** principle.

Q.2- 7 MARKS

1) Give the difference between stack and queue.

Basis of Comparison	Stack	Queue
Definition	A linear data structure that follows LIFO principle	A linear data structure that follows FIFO principle
Principle	LIFO (Last In First Out)	FIFO (First In First Out)
Insertion Operation	Push	Enqueue
Deletion Operation	Pop	Dequeue
Used for Insertion	Top	Rear
Used for Deletion	Top	Front
Number of Pointers	One pointer (Top)	Two pointers (Front and Rear)
Order of Removal	Last inserted element is removed first	First inserted element is removed first
Implementation	Can be implemented using Array or Linked List	Can be implemented using Array or Linked List
Example (Real Life)	Stack of plates	Queue at ticket counter
Applications	Expression evaluation, Parenthesis	CPU scheduling, Printer queue, BFS

Basis of Comparison	Stack	Queue
	checking, Function calls	traversal
Data Access	Only top element can be accessed	Front element is accessed first
Memory Usage	Simple structure	Slightly complex due to two pointers

2) What are the operations of the stack? Describe below.

a. isfull (), b. Push (), c. Pop (), d. Top (or peek) e. IsEmpty ()

OR

What is a Stack? Explain the Various Operations Performed on a Stack in Detail.

OR

Explain stack in depth

☐ Introduction

A **Stack** is a linear data structure in which insertion and deletion of elements take place at only one end, called the **TOP**. It follows the **LIFO (Last In First Out)** principle, which means the element inserted last is removed first. The operations of stack useful in expression evaluation, recursion, and many other applications.

☐ Representation of Stack

A stack can be implemented using:

1. **Array** (Static implementation)
2. **Linked List** (Dynamic implementation)

In array implementation, a variable called **top** is used to keep track of the last inserted element.

□ 1) isFull()

This operation checks whether the stack is full or not.

Condition (Array Implementation):

```
if (top == MAX - 1)
```

Stack is Full

If the stack is full and we try to insert an element, it causes **Overflow condition**.

□ 2) Push()

Push operation is used to insert an element into the stack.

Algorithm:

1. Check if stack is full (isFull).
2. If full → Display "Stack Overflow".
3. Otherwise →
 - Increment top by 1
 - Insert element at stack[top]

Example:

If stack contains: 10, 20

After Push(30) → 10, 20, 30

□ 3) Pop()

Pop operation is used to remove the top element from the stack.

Algorithm:

1. Check if stack is empty (isEmpty).
2. If empty → Display "Stack Underflow".
3. Otherwise →
 - Store stack[top] in a variable
 - Decrement top by 1

Example:

If stack contains: 10, 20, 30

After Pop() → 10, 20

□ 4) Top() or Peek()

Peek operation returns the top element of the stack without removing it.

Condition:

```
if (top != -1)
```

```
    return stack[top]
```

It only displays the top element.

□ 5) isEmpty()

This operation checks whether the stack is empty.

Condition:

```
if (top == -1)
```

```
    Stack is Empty
```

If we try to pop from empty stack, it causes **Underflow condition**.

3) Discuss the various types of data structures with classification into linear and non linear types.

□ Introduction

A **Data Structure** is a method of organizing and storing data in memory so that it can be accessed and modified efficiently.

Data structures are broadly classified into:

- 1 □ Linear Data Structures
- 2 □ Non-Linear Data Structures

□ 1. Linear Data Structures

In **Linear Data Structures**, data elements are arranged in a **sequential (linear) order**.

□ Characteristics:

- Elements are stored in a linear sequence.
- Traversal is done in a single direction.
- Implementation is simple and easy.
- Memory representation may be contiguous or linked.

□ Types of Linear Data Structures:

- **Array** – Collection of similar elements stored in contiguous memory, accessed by index.
- **Stack** – Follows **LIFO** principle; operations: Push and Pop.
- **Queue** – Follows **FIFO** principle; operations: Enqueue and Dequeue.
- **Linked List** – Collection of nodes containing data and pointer; dynamic size.

□ 2. Non-Linear Data Structures

In **Non-Linear Data Structures**, elements are **not arranged sequentially**. One element can be connected to multiple elements.

□ Characteristics:

- Data is arranged hierarchically or in network form.
- Multiple levels of relationships.
- More complex structure compared to linear structures.
- Suitable for representing real-world relationships.

□ Types of Non-Linear Data Structures:

(a) Tree

- Hierarchical structure with root node.
- Consists of parent and child nodes.
- Example: Binary Tree.
- Used in file systems and databases.

(b) Graph

- Collection of vertices (nodes) and edges.
- Represents network relationships.
- Used in social networks, maps, routing algorithms.

□ Difference Between Linear and Non-Linear Data Structures

Linear Data Structure	Non-Linear Data Structure
Elements arranged sequentially	Elements arranged hierarchically
Single level structure	Multi-level structure
Easy to implement	Complex to implement
Example: Array, Stack, Queue	Example: Tree, Graph

4) Define best case, worst case, and average case time complexity.

☐ Introduction

Time Complexity is a measure of the time required by an algorithm to execute as a function of input size **n**.

It helps in comparing algorithms and determining their efficiency.

Time complexity is usually expressed using **Big O notation** such as $O(1)$, $O(n)$, $O(n^2)$, etc.

There are three important cases to analyze an algorithm:

1. Best Case
 2. Worst Case
 3. Average Case
-

☐ 1 ☐ Best Case Time Complexity

The **Best Case** is the minimum number of operations performed by an algorithm for any input of size n .

It represents the most favorable situation.

Example: Linear Search

If the element is found at the first position, only one comparison is needed.

☐ Best Case = **$O(1)$**

☐ 2 ☐ Worst Case Time Complexity

The **Worst Case** is the maximum number of operations performed by an algorithm for any input of size n .

It represents the least favorable condition.

Example: Linear Search

If the element is at the last position or not present in the list, all elements must be checked.

□ Worst Case = **$O(n)$**

Worst case is most commonly considered in algorithm analysis because it guarantees maximum execution time.

□ 3 □ Average Case Time Complexity

The **Average Case** is the expected number of operations performed by an algorithm for all possible inputs of size n .

It gives the overall performance of the algorithm.

Example: Linear Search

On average, the element may be found in the middle of the list.

Approximately $n/2$ comparisons are required.

□ Average Case = **$O(n)$**

□ Comparison Table

Case	Definition	Represents	Example (Linear Search)
Best Case	Minimum time taken	Most favorable condition	$O(1)$
Worst Case	Maximum time taken	Least favorable condition	$O(n)$
Average Case	Expected time taken	Normal performance	$O(n)$

6) Explain the concept of a circular queue. How does it solve the limitations of a linear queue? Write algorithms for insertion and deletion in a circular queue.

□ Introduction

A **Circular Queue** is a linear data structure in which the last position is connected back to the first position to form a circle.

In a normal (linear) queue, once the rear reaches the end of the array, no more elements can be inserted even if there is empty space at the beginning. This leads to **memory wastage**.

Circular queue solves this problem by reusing empty spaces.

□ How Circular Queue Solves Limitation of Linear Queue

□ Limitation of Linear Queue:

- After several deletions, empty spaces at the beginning cannot be reused.
- Causes **false overflow** condition.

□ Solution by Circular Queue:

- When rear reaches the last index, it moves to index 0 if space is available.
- Prevents memory wastage.
- Fully utilizes the array space.

Algorithms for insertion and deletion in a circular queue.

□ Assumptions

- Circular Queue is implemented using array `CQ [MAX]`
- `front` and `rear` are initialized as:

`front = -1`

`rear = -1`

□ 1 □ Algorithm for Insertion (Enqueue)

Step 1: If ($(\text{rear} + 1) \% \text{MAX} == \text{front}$)

 Print "Queue Overflow"

 Stop

Step 2: If (front == -1) // Queue is empty

front = 0

rear = 0

Else

rear = (rear + 1) % MAX

Step 3: CQ[rear] = item

Step 4: Stop

✓ Condition for Full Queue:

(rear + 1) % MAX == front

□ 2 □ **Algorithm for Deletion (Dequeue)**

Step 1: If (front == -1)

Print "Queue Underflow"

Stop

Step 2: item = CQ[front]

Step 3: If (front == rear) // Only one element

front = -1

rear = -1

Else

front = (front + 1) % MAX

Step 4: Return item

Step 5: Stop

7) Explain how queues can be implemented using linked lists. Write algorithms for insertion (enqueue) and deletion (dequeue) operations.

□ 1. Introduction

A **Queue** is a linear data structure that follows the **FIFO (First In First Out)** principle.

In a linked list implementation, the queue is represented using **nodes**, and we maintain two pointers:

- **Front** → Points to the first node
- **Rear** → Points to the last node

This method overcomes the size limitation of array implementation.

□ 2. Structure of Node

Each node contains:

- Data
- Pointer to next node

struct node

```
{  
    int data;  
    struct node *next;  
};
```

□ 3. Enqueue Operation (Insertion)

New element is inserted at the **rear**.

✓ Algorithm: Enqueue

Step 1: Create a new node.

Step 2: If new node is NULL

Print "Overflow"

Stop

Step 3: Set newnode->data = item

Step 4: Set newnode->next = NULL

Step 5: If front == NULL

front = rear = newnode

Else

rear->next = newnode

rear = newnode

Step 6: Stop

□ 4. Dequeue Operation (Deletion)

Element is removed from the **front**.

✓ Algorithm: Dequeue

Step 1: If front == NULL

Print "Underflow"

Stop

Step 2: temp = front

Step 3: item = temp->data

Step 4: front = front->next

Step 5: If front == NULL

rear = NULL

Step 6: Free temp

Step 7: Return item

Q.3 7 MARKS

1) Explain circular linked list and its implementation.

☐ Introduction

A **Circular Linked List (CLL)** is a type of linked list in which the **last node points back to the first node** instead of pointing to NULL.

Thus, the list forms a circular structure. There is no NULL at the end of the list.

☐ Structure of Circular Linked List

Structure in C:

```
struct node {  
    int data;  
    struct node* next;  
};
```

In a circular linked list:

Last Node → next = First Node

☐ Features of Circular Linked List

- No NULL pointer at the end.

- Traversal can start from any node.
 - Efficient for applications where data needs to be accessed repeatedly in a cycle.
 - Can be singly circular or doubly circular.
-

□ Operations in Circular Linked List

□ 1 □ Creation of Circular Linked List

Steps:

1. Create a new node.
 2. If list is empty:
 - $\text{head} = \text{newNode}$
 - $\text{newNode} \rightarrow \text{next} = \text{head}$
 3. Else:
 - Traverse to last node.
 - $\text{last} \rightarrow \text{next} = \text{newNode}$
 - $\text{newNode} \rightarrow \text{next} = \text{head}$
-

□ 2 □ Insertion at Beginning

1. Create new node.
2. Traverse to last node.
3. $\text{newNode} \rightarrow \text{next} = \text{head}$
4. $\text{last} \rightarrow \text{next} = \text{newNode}$
5. $\text{head} = \text{newNode}$

□ 3 □ Insertion at End

1. Create new node.
2. Traverse to last node.
3. $\text{last} \rightarrow \text{next} = \text{newNode}$
4. $\text{newNode} \rightarrow \text{next} = \text{head}$

□ 4 □ Traversal

Traversal is done using a loop until we reach head again.

Example:

```
temp = head;
do {
    printf("%d", temp->data);
    temp = temp->next;
} while(temp != head);
```

2) Explain Recursion.

□ Introduction

Recursion is a method of solving a problem in which a function calls itself repeatedly until a stopping condition is reached.

It is used when a problem can be divided into smaller sub-problems of the same type.

□ **Basic Structure of Recursion**

Every recursive function must contain two important parts:

1. Base Case

- The condition that stops the recursion.
- Prevents infinite function calls.

2. Recursive Case

- The part where the function calls itself with a smaller value.

□ **General Form of Recursive Function**

```
function()
{
    if(base condition)
        return value;    // Base Case
    else
        return function(smaller input); // Recursive Case
}
```

□ **Example: Factorial Using Recursion**

Mathematical Definition:

$$n! = n \times (n-1)! \quad n! = n \times (n-1)! \quad n! = n \times (n-1)!$$

C Program:

```
int factorial(int n)
{
```

```
if(n == 0 || n == 1) // Base Case
    return 1;
else
    return n * factorial(n-1); // Recursive Call
}
```

Working of factorial(4):

factorial(4)
= $4 \times \text{factorial}(3)$
= $4 \times 3 \times \text{factorial}(2)$
= $4 \times 3 \times 2 \times \text{factorial}(1)$
= $4 \times 3 \times 2 \times 1$
= 24

☐ **Advantages**

- Code becomes simple and easy to understand.
- Suitable for problems like:
 - Factorial
 - Fibonacci series
 - Tower of Hanoi
 - Tree traversal

☐ **Disadvantages**

- Uses extra memory (stack).
- Slower than iteration in some cases.
- Risk of stack overflow if base case is missing.

3) What is data structure? What are various types of data structure? Explain.

☐ Introduction

A **Data Structure** is a systematic way of organizing, storing, and managing data in memory so that it can be accessed and modified efficiently.

Efficient data structures help in performing operations such as searching, sorting, insertion, and deletion quickly.

☐ Definition

A data structure is a particular way of arranging data in a computer so that it can be used effectively.

☐ Classification of Data Structures

Data structures are mainly classified into:

1 ☐ **Primitive Data Structures**

2 ☐ **Non-Primitive Data Structures**

☐ 1 ☐ **Primitive Data Structures**

Primitive data structures are the basic data types provided by a programming language.

Examples:

- Integer (int)
- Float (float)
- Character (char)
- Double (double)

- Boolean

These store single values and are directly supported by the machine.

□ 2 □ **Non-Primitive Data Structures**

Non-primitive data structures are derived from primitive data types. They can store multiple values.

Non-primitive data structures are further classified into:

A) Linear Data Structures

In linear data structures, elements are arranged in a sequential order. Each element has a unique predecessor and successor (except first and last).

Examples:

a) Array

- Collection of similar elements
- Stored in contiguous memory

b) Stack

- Follows LIFO principle
- Operations: Push and Pop

c) Queue

- Follows FIFO principle
- Operations: Enqueue and Dequeue

d) Linked List

- Collection of nodes
 - Each node contains data and pointer
-

B) Non-Linear Data Structures

In non-linear data structures, elements are not arranged sequentially. A single element can be connected to multiple elements.

Examples:

a) Tree

- Hierarchical structure
- Consists of root and child nodes

b) Graph

- Collection of vertices and edges
- Represents network relationships

□ Difference Between Linear and Non-Linear Data Structures

Linear Data Structure	Non-Linear Data Structure
Sequential arrangement	Hierarchical/network arrangement
Single level	Multi-level
Easy to implement	Complex structure
Example: Array, Stack	Example: Tree, Graph

4) What are the applications of stack? Describe below. The following are the applications of stacks

- **Balancing the parenthesis**
- **Expression Evaluation**
- **Function call management**

A Stack is a linear data structure that follows the LIFO (Last In First Out) principle. Due to its LIFO nature, stack is widely used in many computer science applications.

□ 1. Balancing the Parenthesis

Stack is used to check whether an expression has **balanced parentheses**.

□ Example:

Expression:

$(A + B) * (C - D)$

□ Algorithm Idea:

1. Scan the expression from left to right.
2. If opening bracket (, {, [→ **Push** into stack.
3. If closing bracket), },] →
 - Pop from stack
 - Check if it matches the opening bracket
4. If stack becomes empty at the end → Expression is balanced.

✓ Used in compilers and syntax checking.

□ 2. Expression Evaluation

Stack is used in evaluating **Postfix and Prefix expressions**.

□ Postfix Example:

$2\ 3 + 5 *$

□ **Method:**

1. Scan expression from left to right.
2. If operand → **Push** into stack.
3. If operator →
 - Pop two operands
 - Perform operation
 - Push result back

✓ Used in calculators and compiler design.

□ **3 □ Function Call Management (Recursion)**

When a function is called, its:

- Local variables
- Parameters
- Return address

are stored in a **Call Stack**.

□ **Example:**

If function A calls B, and B calls C:

Stack will store:

C

B

A

When C finishes → it is removed first (LIFO).

This process is called **Stack Frame Management**.

✓ Used in recursion and program execution.

Q.4 7 MARKS

1) Explain Sparse Matrix and its Representation.

☐ Introduction

A **Sparse Matrix** is a matrix in which most of the elements are **zero** and only a few elements are non-zero. When the number of zero elements is greater than the number of non-zero elements, the matrix is called a sparse matrix.

☐ Example of Sparse Matrix

0 0 3 0

0 0 0 0

5 0 0 0

0 0 0 2

☐ Analysis:

Total elements = $4 \times 4 = 16$

Non-zero elements = 3 (3, 5, 2)

Zero elements = 13

Since zeros are more than non-zeros → It is a **Sparse Matrix**.

Need for Sparse Matrix Representation

Storing all elements (including zeros) wastes memory.

To save memory, only **non-zero elements** are stored.

Representation of Sparse Matrix

There are mainly two methods:

- 1 □ Triplet (3-Tuple) Representation
- 2 □ Linked List Representation

□ 1 □ Triplet (3-Tuple) Representation

In this method, only non-zero elements are stored in the form of:

- Row number
- Column number
- Value

Format:

Row Column Value

The first row of the table stores:

- Total rows
- Total columns
- Total non-zero elements

Example:

For the previous matrix:

Row Column Value

3	3	2
0	2	5
2	0	8

Here:

- 3 rows
- 3 columns

- 2 non-zero elements

□ This method saves memory by storing only useful data.

□ 2 □ **Linked List Representation**

In this method:

- Each node stores:
 - Row index
 - Column index
 - Value
 - Pointer to next node

Nodes are linked together to form a list of non-zero elements.

This method is dynamic and efficient for large sparse matrices.

3) Introduction to dynamic memory allocation and explain DMA function malloc (), calloc () & free ().”

Introduction

Dynamic Memory Allocation (DMA) is the process of allocating memory at **run time** (during program execution).

In C language, memory can be allocated dynamically from the **heap memory** using special library functions.

□ DMA is useful when:

- Size of data is not known in advance
- Memory requirement changes during execution
- Working with linked lists, trees, etc.

To use DMA functions, we include:

```
#include <stdlib.h>
```

DMA Functions in C

The main DMA functions are:

1. `malloc()`
2. `calloc()`
3. `free()`
4. `realloc()` (optional)

❑ 1 ❑ **malloc() Function**

➤ **Meaning:**

`malloc` stands for **Memory Allocation**.

➤ **Definition:**

It allocates a single block of memory of specified size.

➤ **Syntax:**

```
ptr = (data_type*) malloc(size_in_bytes);
```

➤ **Example:**

```
int *ptr;
```

```
ptr = (int*) malloc(5 * sizeof(int));
```

❑ 2 ❑ **calloc() Function**

➤ **Meaning:**

`calloc` stands for **C**ontiguous **A**llocation.

➤ **Definition:**

It allocates multiple blocks of memory and initializes them to zero.

➤ **Syntax:**

```
ptr = (data_type*) calloc(number_of_elements, size_of_each_element);
```

➤ **Example:**

```
int *ptr;  
  
ptr = (int*) calloc(5, sizeof(int));
```

❑ **Difference Between malloc() and calloc()**

malloc()	calloc()
Allocates single block	Allocates multiple blocks
Does not initialize memory	Initializes memory to zero
Takes 1 argument	Takes 2 arguments
Faster than calloc	Slightly slower

❑ 3 ❑ **free() Function**

➤ **Definition:**

`free()` is used to release dynamically allocated memory.

➤ **Syntax:**

```
free(ptr);
```

➤ **Example:**

free(ptr);

4) Evaluate the Postfix expression in Tabular Forms : 2 5 3 - * 8 / 4 +

□ Step-by-Step Evaluation Using Stack (Tabular Form)

Step	Symbol	Operation Performed	Stack After Operation
1	2	Push 2	2
2	5	Push 5	2, 5
3	3	Push 3	2, 5, 3
4	-	$5 - 3 = 2$	2, 2
5	*	$2 \times 2 = 4$	4
6	8	Push 8	4, 8
7	/	$4 \div 8 = 0.5$	0.5
8	4	Push 4	0.5, 4
9	+	$0.5 + 4 = 4.5$	4.5

□ Final Answer

4.5\boxed{4.5}4.5

□ Explanation

Postfix expression is evaluated using **Stack**:

- If operand → **Push** into stack
- If operator →
 - Pop two elements
 - Perform operation
 - Push result back

5) Explain Tower of Hanoi with Suitable Example.

□ 1. Introduction

Tower of Hanoi is a famous mathematical puzzle and a classic example used to explain **recursion** in Data Structures.

It was invented by the French mathematician **Édouard Lucas** in 1883.

□ 2. Rules of Tower of Hanoi

1. Only **one disk** can be moved at a time.
 2. Only the **top disk** of a rod can be moved.
 3. A **larger disk cannot be placed on a smaller disk**.
-

□ 3. Objective

Move all disks from **Source rod (A)** to **Destination rod (C)** using **Auxiliary rod (B)**.

□ 4. Example: 3 Disks

Suppose we have 3 disks:

Source = A

Auxiliary = B

Destination = C

Steps:

1. Move Disk 1 from A $\rightarrow$ C
2. Move Disk 2 from A $\rightarrow$ B
3. Move Disk 1 from C $\rightarrow$ B

4. Move Disk 3 from A $\rightarrow$ C

5. Move Disk 1 from B $\rightarrow$ A

6. Move Disk 2 from B $\rightarrow$ C

7. Move Disk 1 from A $\rightarrow$ C

✓ Total Moves = 7

□ 7. Time Complexity

Time Complexity = $O(2^n)$

It increases exponentially.

□ 8. Applications

- Understanding recursion
 - Algorithm design concept
 - Problem solving using divide and conquer
-